🍄 SporeERP — Implementation Status & Developer Handover

Document 11: What Is Built (Sprints 1–3), How It Works, and How to Continue

Project Name: SporeERP **Document Code:** DOC-11-IMPL-STATUS **Branch:** `initial/codesetup` **Last Updated:** 2026-10-14 **Covers:** Sprint 1 (Infra/DB/Scaffolding), Sprint 2 (Auth, Profiles & RBAC) and Sprint 3 (Catalog & Storefront) from [00_MASTER_PROJECT_PLAN_AND_ROADMAP.md] (00_MASTER_PROJECT_PLAN_AND_ROADMAP.md) > Read this first if you are picking up the codebase. It records what is actually implemented (not just planned), the decisions that differ from the spec documents, and the rules to follow when adding features. ---

1. Tech Stack (as installed)

| Concern | Choice | Notes | | :--- | :--- | :--- | | Framework | **Next.js 16.4** (App Router, Turbopack) | `cacheComponents` + `partialPrefetching` enabled in `next.config.ts` — see §6.3 | | UI | React 19.3, Tailwind CSS v4 | Design tokens in `app/globals.css` (forest/ivory/charcoal palette) | | Language | TypeScript 5 (strict) | Path alias `@/*` → project root | | ORM / DB | Prisma 6.19 → **MariaDB 11** (provider `mysql`) | Schema in `prisma/schema.prisma`, migrations in `prisma/migrations/` | | Auth | Auth.js / NextAuth **v5 beta** | Credentials provider, JWT sessions | | Validation | Zod 4 | Shared schemas in `lib/validations/` | | Hashing | bcryptjs, cost factor **12** | `lib/auth/password.ts` | | Infra (dev) | Docker Compose: MariaDB 11, Redis 7, Adminer | `docker-compose.yml` (optional — a local MariaDB install works too) | | Not yet used | ioredis, razorpay | Installed for later sprints | ---

2. Local Setup (from a fresh clone)

```
bash
npm install
cp .env.example .env          # then fill in the values below
```

2.1 Database

You need a **MariaDB** server (not MySQL 8 — see §8 troubleshooting). Either: - **Docker:** `docker compose up -d` (MariaDB on `3306`, Redis on `6379`, Adminer on `8080`), or - **Local install:** create the database and user once as root (on Windows with both MySQL 8 and MariaDB installed, MariaDB is typically on another port such as `3308` — connect there, see §9):

```
sql
CREATE DATABASE IF NOT EXISTS sporeerp CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER IF NOT EXISTS 'sporeerp'@'localhost' IDENTIFIED BY '<your password>';
GRANT ALL PRIVILEGES ON sporeerp.* TO 'sporeerp'@'localhost';
-- `prisma migrate dev` creates a temporary shadow database, which needs these:
GRANT CREATE, ALTER, DROP, REFERENCES ON *.* TO 'sporeerp'@'localhost';
FLUSH PRIVILEGES;
```

2.2 `.env` keys

| Key | Purpose | | :--- | :--- | | `DATABASE_URL` | `mysql://sporeerp:<pwd>@localhost:<port>/sporeerp?connection_limit=10&pool_timeout=20` — use the port **your MariaDB** listens on | | `REDIS_URL` | Reserved (rate limiting / caching — not wired yet) | | `AUTH_SECRET` | **Required.** Generate with `npx auth secret` or `openssl rand -base64 32` | | `NEXTAUTH_URL` | Base URL used in reset/invite links (`http://localhost:3000` locally) | | `AUTH_TRUST_HOST` | `true` — required for `next start` / self-hosting behind Nginx | | `SEED_ADMIN_NAME` / `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD` | Founder admin account created by the seed (skipped if empty) | | `RAZORPAY_*` | Sprint 5 |

2.3 Apply migrations, seed, run

```
bash
npm run prisma:migrate:deploy   # applies every migration in prisma/migrations
npm run db:seed                 # roles, permissions, role grants, founder admin, base data
npm run dev                     # http://localhost:3000
```

Sign in at `/login` with `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD` → you land on `/admin/dashboard`.

2.4 npm scripts

| Script | What it does | | :--- | :--- | | `dev` / `build` / `start` / `lint` | Standard Next.js | | `prisma:generate` | Regenerate Prisma Client | | `prisma:migrate` | `prisma migrate dev` — create + apply a new migration from schema changes | | `prisma:migrate:create` | Create a migration file **without** applying it (to review/edit SQL first) | | `prisma:migrate:deploy` | Apply pending migrations (CI / production / fresh clones) | | `prisma:migrate:status` | Show which migrations are applied | | `prisma:studio` | Prisma Studio DB browser | | `db:seed` | Run `prisma/seed.ts` (idempotent — safe to re-run) | ---

3. Database Migrations Policy

Never use `prisma db push`. Every schema change ships as a versioned migration file so every environment stays in sync. | Migration | Contents | | :--- | :--- | | `20261014000000_init` | Full Sprint 1 schema: all 29 models / 14 enums from DOC-04 | | `20261014000100_add_password_reset_tokens` | Sprint 2: `password_reset_tokens` table (reset + staff invite links) | | `20261014000200_add_catalog_pricing_tiers_and_product_details` | Sprint 3:

`product_pricing_tiers`, `stock_notifications`; product columns `short_description`, `variety` (indexed), `nutrition_info`, `storage_tips`, `is_featured`, `allow_backorder`; drops `products.b2b_price` (replaced by tiers) | **Workflow for a schema change:** 1. Edit `prisma/schema.prisma`. 2. `npm run prisma:migrate -- --name <short_description>` (e.g. `add_product_pricing_tiers`). 3. Review the generated SQL in `prisma/migrations/<timestamp>_<name>/migration.sql`. 4. Commit the schema **and** the migration folder together. 5. Teammates pull and run `npm run prisma:migrate:deploy`. > **Migration order = folder name order.** `prisma migrate dev` names folders with the current **UTC** timestamp. The existing folders are stamped `20261014…`, so a migration created before **2026-10-14 00:03 UTC** would sort *before* them and break fresh installs. If that happens, rename the folder so it sorts last (and update `_prisma_migrations.migration_name` in your local DB). Verify with a fresh replay: create an empty database, `migrate deploy` into it, then `prisma migrate diff --from-url <that db> --to-schema-datamodel prisma/schema.prisma --exit-code`. If a database was previously created with `db push`, baseline it once: `npx prisma migrate resolve --applied 20261014000000_init`, then `migrate deploy`. ---

4. Project Structure

```
auth.ts                      Auth.js instance (Credentials provider, DB lookups) → handlers/auth
auth.config.ts               DB-free Auth.js config (JWT/session callbacks) — shared with proxy.
proxy.ts                     Route guard (Next 16's replacement for middleware.ts)
types/next-auth.d.ts         Adds `id` + `roles` to Session / JWT types

app/
  (customer)/                Public storefront + customer account (route group, ivory theme)
    page.tsx                 Landing page (11 sections, see §7.3.1); copy in home-content.ts
    products/                Catalog with filters (page.tsx), product detail page ([slug]/), not
    login/ register/ forgot-password/ reset-password/ account/
    loading.tsx              Suspense fallback for the group
  admin/                     Staff ERP (charcoal theme)
    layout.tsx               Sidebar with permission-filtered nav
    dashboard/               Placeholder dashboard (shows user + roles)
    employees/               Staff management (invite / roles / deactivate)
    products/                Product list + filters, new/, [id]/ edit + archive, product-form.ts
    categories/              Category tree (two levels) create/edit
  unauthorized/              "Access denied" page
  actions/auth.ts            Server Actions for login/register/reset/profile/sign-out
  api/
    health/                  GET  → DB ping latency
    auth/[...nextauth]/      Auth.js endpoints (session, callback, csrf, signout…)
    auth/register/           POST → register customer (JSON API)
    auth/forgot-password/    POST → request reset link (JSON API)
    v1/products/             GET (public catalog) / POST (create product, catalog:product:manage
  uploads/[...path]/         Serves uploaded images from storage/uploads (immutable cache)
  sitemap.ts  robots.ts      SEO files

lib/
  db.ts                      Prisma singleton
  auth/permissions.ts        ROLES, PERMISSIONS, ROLE_PERMISSIONS matrix (source of truth for se
  auth/dal.ts                Data Access Layer: getCurrentUser / requireUser / requireStaff / re
  auth/password.ts           bcrypt hash/verify, secure token generation + SHA-256 hashing
  validations/auth.ts        Zod schemas for every auth/employee form & API
  api-response.ts            JSON success/error envelopes (DOC-05 §1)
  form-state.ts              Server Action form state helpers, safe callbackUrl check
  errors.ts                  ServiceError (known, user-facing failures)
```

```
    mail.ts                    Mail stub — logs emails to the server console (see §8)
    url.ts                     appUrl()/siteUrl() — absolute URLs from NEXTAUTH_URL
    settings.ts                Cached store settings (name, contact details, service area) from bu
    storage.ts                 Image upload: content sniffing, size limits, save/delete under stor
    auth/api.ts                authorizeApi(permission) — 401/403 envelopes for JSON routes (no re
    catalog/format.ts          Unit labels, slugify, SKU generator (client-safe)
    catalog/queries.ts         Cached ("use cache", tag "catalog") storefront reads returning plai
    validations/catalog.ts     Zod schemas: product form/API, list query, category, notify-me

  services/                    Business logic (no HTTP / React here)
    auth.service.ts            register, password reset, profile, change password
    employee.service.ts        list/invite employees, assign roles, activate/deactivate
    catalog.service.ts         categories, product create/update/archive, images, tiers, notify-me
    audit.service.ts           Writes audit_logs rows

  components/
    ui/button.tsx
    forms/form-controls.tsx    Field / TextArea / Select / Checkbox / SubmitButton / FormMessage (
    forms/auth-card.tsx
    auth/user-nav.tsx          Header sign-in state (server, inside Suspense)
    auth/admin-nav.tsx         Admin sidebar links + sign out (server, inside Suspense)
    catalog/product-card.tsx   ProductCard, ProductImage, AvailabilityBadge
    catalog/status-badge.tsx   Admin product status pill
    storefront/site-header.tsx / site-footer.tsx   Storefront header (sticky, mobile menu) and fo

  storage/uploads/             Uploaded media (gitignored; back it up together with the database)

  prisma/
    schema.prisma  migrations/  seed.ts
```

Layering rule: pages / Server Actions / API routes → validate with Zod → check auth via `lib/auth/dal.ts` → call a function in `services/` → services use `db` (Prisma). Services throw `ServiceError` for expected failures; callers convert them with `serviceFailure()` (forms) or `apiErrorFrom()` (JSON APIs). ---

5. Sprint 1 — Implemented

- [x] Next.js + TypeScript + ESLint + Prettier, `@/*` alias - [x] Docker Compose: MariaDB 11 (utf8mb4), Redis 7, Adminer - [x] Prisma configured with pool params in `DATABASE_URL` - [x] Full relational schema (DOC-04) — now delivered as migration `20261014000000_init` - [x] Route layout: `app/(customer)`, `app/admin`, `app/api`, `lib/`, `services/`, `types/` - [x] `GET /api/health` → `{ status, database, dbLatencyMs }` (503 if DB is down) - [x] Seed: `store_name` setting + "Fresh Mushrooms" category ---

6. Sprint 2 — Authentication & RBAC (Implemented)

6.1 Roles & permissions

Defined in `lib/auth/permissions.ts` and written to `roles`, `permissions`, `role_permissions` by `npm run db:seed`. Re-running the seed re-syncs each role's grants with the matrix. | Role | Granted permissions | | :--- | :--- | | `FOUNDER_ADMIN` | All 24 | | `FARM_MANAGER` | users:view, catalog:product:view/manage, orders:view_all/update_status/cancel_refund, inventory:* (3), production:* (3), harvest:* (2), suppliers:manage, purchases:create,

expenses:record/view_all | | `PRODUCTION_STAFF` | catalog:product:view, inventory:view_stock, production:* (3), harvest:* (2) | | `INVENTORY_STAFF` | catalog:product:view, orders:view_all, inventory:* (3), purchases:create | | `OPERATIONS_STAFF` | catalog:product:view, orders:view_all, orders:update_status, inventory:view_stock | | `CUSTOMER` | catalog:product:view, orders:create_own | Staff roles = every role except `CUSTOMER`.

6.2 Authentication flow

- **Credentials login** (`auth.ts`): email is lower-cased, password checked with bcrypt; unknown emails are compared against a dummy hash so timing doesn't leak which emails exist. Deactivated users (`is_active = false`) cannot sign in. - **Session:** signed JWT in an HTTP-only cookie (`__Secure-` prefixed + `Secure` over HTTPS). The token carries `id` and `roles`. - **After login:** redirect to `callbackUrl` if present (only same-origin relative paths are accepted), else staff → `/admin/dashboard`, customers → `/account`. - **Register:** creates `users` + `customers` rows and the `CUSTOMER` role, then signs the user in. - **Forgot password:** always returns the same message (no account enumeration). Creates a single-use token valid **1 hour**; only its SHA-256 hash is stored. - **Reset password:** validates token (exists, unused, unexpired), updates the hash, marks the token used and deletes other open tokens. - **Profile (`/account`):** edit name, phone, B2B company name and GSTIN (customers only); change password (requires current password); sign out.

6.3 Authorization — two layers

1. **`proxy.ts` (optimistic, no DB):** matcher `/admin/:path*`, `/account/:path*`, `/checkout/:path*`, `/api/v1/:path*`. - Not signed in → redirect to `/login?callbackUrl=<path>` (API routes get a `401` JSON envelope instead). - `/admin/*` without a staff role → `/unauthorized`. - `/admin/reports*`, `/admin/settings*` without `FOUNDER_ADMIN` → `/admin/dashboard?error=restricted`. 2. **`lib/auth/dal.ts` (authoritative, hits DB):** **`getCurrentUser()` loads the user, active flag, roles and permissions **fresh from the database on every request** (deduped per render with React `cache`). Use: - `requireUser(returnTo)` — any signed-in, active user - `requireStaff(returnTo)` — staff role required - `requirePermission(PERMISSIONS.X, returnTo)` — specific permission required - `hasPermission(user, PERMISSIONS.X)` — conditional UI **Every admin page and every Server Action must call one of these** — the proxy alone is not a security boundary (Server Actions are reachable directly). Because permissions are read from the DB each request, **role/permission edits and deactivations take effect on the user's next request with no redeploy**. **Next.js 16 + Cache Components rule:** anything that reads the session (`auth()`, `getCurrentUser()`, `searchParams`) must render inside a `<Suspense>` boundary. Each route group has a `loading.tsx` for pages; shared chrome (header/sidebar) wraps its session-aware piece in `<Suspense>` (see `components/auth/*`). Don't `await` the session at the top of a layout. Read the session only via `getCurrentUser()` — it calls `await connection()` before `auth()`, otherwise dev mode reports Auth.js's `crypto.getRandomValues()` as an unstable prerender value. Values like `new Date()` in static UI need `"use cache"` (see the footer year in `app/(customer)/layout.tsx`).

6.4 Employee management (`/admin/employees`)

| Action | Permission | Behaviour | | :--- | :--- | :--- | | View staff list | `users:view` (Founder, Farm Manager) | Name, email, designation/department/joining date, roles, status | | Invite staff | `users:manage_roles` (Founder) | Creates `users` (random unusable password) + `employees` + roles, issues a **72-hour** set-password link (emailed + shown on screen to share) | | Assign roles | `users:manage_roles` | Replaces the user's staff roles; non-staff roles untouched | | Deactivate / reactivate | `users:manage_roles` | Flips `users.is_active` and `employees.is_active`; deactivated users are locked out immediately | Guards: you cannot remove your own `FOUNDER_ADMIN` role or deactivate yourself. All three mutations write an `audit_logs` row (`EMPLOYEE_INVITED`, `ROLES_ASSIGNED`, `EMPLOYEE_DEACTIVATED` / `EMPLOYEE_REACTIVATED`) with old/new state.

6.5 JSON API endpoints

| Method | Endpoint | Body | Responses | | :--- | :--- | :--- | :--- | | `POST` | `/api/auth/register` | `{ name, email, phone?, password, confirmPassword }` | `201` user · `409 EMAIL_TAKEN/PHONE_TAKEN` · `422 VALIDATION_ERROR` | | `POST` | `/api/auth/forgot-password` | `{ email }` | `200` generic message · `422` | | `GET` | `/api/auth/session` | — | Auth.js session (`user.id`, `user.roles`) or `null` | | `POST` | `/api/auth/callback/credentials` | Auth.js form post | Handled by Auth.js | Envelopes follow DOC-05 §1: `{ success, data, meta: { timestamp, requestId } }` / `{ success: false, error: { code, message, details } }`.

6.6 Password rules

8–72 characters, at least one letter and one number (`lib/validations/auth.ts`). 72 is bcrypt's input limit. ---

7. Sprint 3 — Catalog & Storefront (Implemented)

7.1 Data model

- **`ProductPricingTier`** — one row per product per B2B customer type (`B2B_RESTAURANT`, `WHOLESALE`) with `price` and `minQty` (the quantity from which it applies). Retail price stays on `products.retail_price`. The API's `b2bPrice` field (DOC-05) is derived from the Restaurant tier. - **`StockNotification`** — "notify me when back in stock" requests, unique per product + email. - **Product additions:** `shortDescription` (cards), `variety` (filter, e.g. "Blue Oyster"), `nutritionInfo`, `storageTips` (product page tabs), `isFeatured` (homepage), `allowBackorder`. - **Stock** uses the existing `inventory_items` table: every product gets a linked `FINISHED_PRODUCT` inventory item. Sellable stock = `available_stock − reserved_stock`. **Opening stock** entered on product creation is recorded as a `MANUAL_ADJUSTMENT` inventory transaction. After creation, stock is read-only in the product form (adjustments arrive with the Inventory module, Sprint 5). - **Seeded taxonomy:** Fresh Mushrooms; Processed Products › Dry Mushrooms, Mushroom Powders; Cultivation Supplies › Spawn, Grow Kits.

7.2 Admin screens

| Screen | Permission | What it does | | :--- | :--- | :--- | | `/admin/products` | `catalog:product:view` (all staff) | Search (name/SKU/variety), category & status filters; thumbnail, SKU, price/unit, stock with low-stock flag (≤ reorder level), status | | `/admin/products/new` | `catalog:product:manage` | Full product form (below) | | `/admin/products/[id]` | `catalog:product:manage` | Same form pre-filled + **Archive** | | `/admin/categories` | `catalog:product:manage` | Create/edit categories (two levels), parent, description, active flag | **Product form:** name, category, variety (suggestions from existing ones), SKU (blank → generated, or the **Generate** button), short + full description, selling unit (KG / 200 g / 500 g / pack / bottle), retail price, minimum order quantity, Restaurant + Wholesale tiers (price + from-quantity), opening stock (create only), low-stock level, shelf life, nutrition, cooking & storage, images, status, featured, allow back-orders. **Rules enforced by `services/catalog.service.ts`:** - **SKU** like `MUSH-OYS-BLU-1KG` (category word – variety words – unit); `-2`, `-3` … appended if taken. A manually entered duplicate → `SKU_TAKEN`. - **Slug** from the name, made unique, and **kept on rename** so URLs and SEO don't break (same for category slugs). - **Pricing override:** setting prices on a **new** product needs `catalog:product:manage`; **changing** the retail price or any tier of an **existing** product needs `catalog:pricing:override` (Founder only). Farm Managers see the price fields read-only with a notice; the service also rejects the change server-side. - **Images:** JPEG / PNG / WebP / AVIF, verified by file content (not the extension), ≤ 4 MB each, ≤ 5 per save. Exactly one **main image** (the chosen one, else the existing main, else the first). Removing an image deletes the file only if no other product image still uses that URL. If the database write fails, only the files saved by that request are deleted again. - **Archive instead of delete** — products are referenced by orders, so removal from sale sets `ARCHIVED`. - Every product create/update/archive and category change writes an `audit_logs` row (`PRODUCT_CREATED`, `PRODUCT_UPDATED` with old/new prices, `PRODUCT_ARCHIVED`, `CATEGORY_CREATED`, `CATEGORY_UPDATED`).

7.3 Storefront

| Route | Content | | :--- | :--- | | `/` | Landing page that works even with an empty catalog — see §7.3.1 | | `/products` | Grid with filters: category (incl. sub-categories), variety, in-stock only, search; pagination (24 per page); sort: in stock → featured → name | | `/products/[slug]` | Gallery with thumbnails; category/variety/availability badges; price per unit and minimum order; freshness guarantee (shelf life); quantity stepper (min = MOQ, step 0.5 kg or 1, max = stock unless back-order); business pricing tiers; tabs (Description / Nutrition / Cooking & storage). Out of stock without back-order → **Notify me** form instead | - **Visibility:** customers see `PUBLISHED` and `OUT_OF_STOCK` products in active categories; `DRAFT` / `ARCHIVED` are hidden (direct URL → "Product not found" with `noindex`). - **"In stock"** = status `PUBLISHED` and sellable stock > 0. - **Add to cart** is rendered but disabled ("Online ordering is coming soon") until Sprint 4. #### 7.3.1 Landing page, header & footer `app/(customer)/page.tsx` — sections in order (anchors in brackets are linked from the header and footer): 1. **Hero** — headline, Shop / B2B buttons, **live "TODAY'S HARVEST: X KG AVAILABLE NOW"** (sum of in-stock kg products; "sold out" message at 0), and a "day at the farm" timeline (5 AM harvest, 8–11 AM and 4–7 PM

delivery slots from DOC-07/08). 2. **Highlights strip** — 5:00 AM harvest, < 4 hrs farm to door, 0 pesticides, 100% composted substrate. 3. **Our mushrooms** (`#varieties`) — 6 variety cards (Blue / Pink / King Oyster, White Milky, Paddy Straw, Dry & Powder) with taste and best uses; a card shows **In stock / Sold out** and links to the filtered catalog when products of that variety exist. 4. **Today's harvest** — featured products; with an empty catalog it shows "Our first harvest is on its way" + Create account. 5. **How it works** (`#how-it-works`) — 4 steps from grow room to doorstep. 6. **Our farm** (`#our-farm`) — growing method and checklist. 7. **Nutrition** (`#nutrition`) — 4 benefit cards (no medical claims). 8. **Ways to cook** (`#recipes`) — 3 quick recipe ideas. 9. **For business** (`#business`) — B2B benefits + Register your business. 10. **FAQ** (`#faq`) — 6 questions as an accordion, plus **`FAQPage` JSON-LD** for search engines. The delivery answer uses the `service_area` setting. 11. **Final call to action** — Create free account / Browse products. All homepage copy lives in **`app/(customer)/home-content.ts`** — edit text there without touching the layout. The copy follows the claims in DOC-01/07/08; **founders should review it** (e.g. the variety list, dispatch times and payment methods) before launch. **Header** (`components/storefront/site-header.tsx`): sticky, store name, links (Shop, Varieties, How it works, For business, FAQ), sign-in/account, **Shop now** button, and a ☰ menu on mobile. **Footer** (`components/storefront/site-footer.tsx`): brand + tagline + contact details, **Shop** (all top-level categories from the DB + "In stock today"), **Learn** (section anchors), **Account** (create account, sign in, my account, B2B), bottom bar with © year. **Contact details come from `business_settings`** (seeded empty, so they're hidden until filled in) via `lib/settings.ts`: | Key | Shown as | | :--- | :--- | | `store_name` | Header/footer brand | | `contact_phone` | 📞 `tel:` link | | `contact_whatsapp` | 💬 "Chat on WhatsApp" (`wa.me` link; include the country code) | | `contact_email` | ✉ `mailto:` link | | `contact_address` | 📍 address (multi-line) | | `service_area` | FAQ "Where do you deliver?" answer | There is no settings screen yet (founder settings come later), so set them with **Prisma Studio** (`npm run prisma:studio` → **`BusinessSetting`**) or SQL. Settings are cached with **`cacheLife("minutes")`, so a change appears on the site **within about a minute** (verified: ~70 s). A future settings screen should call `updateTag("settings")` after saving.

7.4 Caching (Next.js Cache Components)

- All storefront reads live in `lib/catalog/queries.ts`, marked `"use cache"`, tagged **`catalog`**, with `cacheLife("minutes")`. They return plain JSON DTOs (Prisma `Decimal` → `number`) because cached values must be serializable. - Admin Server Actions call **`updateTag("catalog")`** after every write, so the storefront shows the change on the very next request. The JSON API uses `revalidateTag("catalog", "max")` (route handlers can't call `updateTag`). - **Any future code that changes stock, prices or product visibility (orders, inventory, harvest) must also invalidate the `catalog` tag**, otherwise the storefront can show stale stock for a few minutes. - Filtering and pagination happen in memory over the visible catalog (tens of products), because "in stock" compares two columns, which Prisma can't express in `WHERE`. Revisit with raw SQL if the catalog grows into the thousands. - The homepage and catalog shells are prerendered at **build time**, so **`next build` needs a reachable database**.

7.5 Images & uploads

- Stored in **`storage/uploads/products/<uuid>.<ext>`** (gitignored) and served by `app/uploads/[...path]/route.ts` with `Cache-Control: immutable` and `nosniff`; path traversal is rejected. They are not under `public/` because `next start` only serves files that were in `public/` at build time. - Rendered through `next/image` (`images.localPatterns` allows `/uploads/**`). - Upload limits: `experimental.serverActions.bodySizeLimit` and `experimental.proxyClientMaxBodySize` are both **22 MB** in `next.config.ts` (the proxy buffers `/admin/*` request bodies too). - **Back up `storage/uploads` together with the database.** Moving to S3/R2 later only requires changing `lib/storage.ts` (save/delete/read) and the image URL prefix.

7.6 SEO

- Root `metadataBase` from `NEXTAUTH_URL`; per-page titles and descriptions; product pages have a canonical URL, OpenGraph + Twitter tags with product images, and **JSON-LD `Product`** with an `Offer` (INR price; `InStock` / `BackOrder` / `OutOfStock`). - `/sitemap.xml` (home, catalog, every visible product with `lastModified`) and `/robots.txt` (disallows `/admin`, `/account`, `/checkout`, `/api/`, `/reset-password`).

7.7 JSON API

| Method | Endpoint | Access | Notes | | :--- | :--- | :--- | :--- | | `GET` | `/api/v1/products` | **Public** (exempted in `proxy.ts`) | Query: `category` (slug), `variety`, `status` (`PUBLISHED` / `OUT_OF_STOCK`), `inStock=1`, `q`, `page`, `limit` (1–100). Returns `products[]` (id, name, slug, sku, category, variety, retailPrice, b2bPrice, pricingTiers, unitType, minOrderQty, availableStock, inStock, shelfLifeDays, primaryImage, status) + `pagination` | | `POST` | `/api/v1/products` | `catalog:product:manage` | Body per DOC-05: `name`, `categoryId`, `description`, `unitType`, `retailPrice`; optional `sku`, `b2bPrice` (→ Restaurant tier) or `pricingTiers[]`, `minOrderQty`, `shelfLifeDays`, `status`, `openingStock`, `images[{imageUrl, isPrimary}]` (`/uploads/…` or `https://`). Responses `201` / `401` / `403` / `409 SKU_TAKEN` / `422` | ---

8. Deviations From the Spec & Known Gaps

| Item | Spec said | What we did / why | | :--- | :--- | :--- | | Session storage | "MariaDB Adapter, database sessions" (DOC-00) | **JWT sessions, no adapter.** Auth.js does not support database sessions with the Credentials provider. Matches DOC-06 ("Stateless Signed JWT + Server-Side Permissions Validation"). | | Middleware | `middleware.ts` (DOC-06 §4) | **`proxy.ts`** — Next.js 16 renamed the convention. Same logic. | | "Read" cells on write permissions | e.g. Inventory Staff "Read" on `harvest:log_entry`, `suppliers:manage` | **Not granted** — a single atomic code can't mean read-only. Add `harvest:view` / `suppliers:view` codes when those modules are built. | | Role promotion in the proxy | — | The proxy reads roles from the JWT, so a **customer promoted to staff must sign in again** before `/admin` opens. Demotions/deactivations are enforced immediately by the DAL. | | Email delivery | Reset/invite emails | **Stub** — `lib/mail.ts` logs the message (including the link) to the server console. Replace with a real provider (SMTP/Resend/SES) before go-live. | | Rate limiting | Redis-backed (DOC-02/06) | **Not implemented yet.** Login / register / forgot-password should get Redis rate limits. | | Email

verification | `users.email_verified` column exists | Not enforced or sent yet. | | Customer addresses | `customer_addresses` table exists | No UI yet — planned with checkout (Sprint 4). | | `/api/auth/session` permissions | DOC-05 says "claims and permissions" | Returns `id` + `roles` only. Server code should use `getCurrentUser().permissions`. | | `products.b2b_price` | DOC-04 column | **Dropped** in favour of `product_pricing_tiers` (one source of truth). API responses still expose `b2bPrice` (= Restaurant tier). | | Permission for product create | DOC-05 says `catalog:product:create` | Uses the matrix code **`catalog:product:manage`** (DOC-06). | | `catalog:pricing:override` | Founder-only; meaning not specified | Interpreted as **changing the prices of an existing product** (see §7.2). | | Image storage / CDN | DOC-05 shows `cdn.sporemushroom.com` URLs | **Local disk** (`storage/uploads`) for now; swap `lib/storage.ts` for S3/R2 + CDN later. | | Product page freshness card | "Picked from Batch BATCH-2026-048 • Harvest Date" | Shows the shelf-life guarantee only; batch traceability needs production data (Sprint 6). | | Add to cart / notify-me emails | Sprint 3 product page | Add to cart disabled until Sprint 4. Notify-me requests are **stored** but no email is sent yet — send them when stock is replenished (Sprint 5 inventory). | | HTTP status of missing products | — | Unknown/archived product URLs render "not found" with `noindex`, but the HTTP status is `200` (the page streams behind `loading.tsx`). | | Category form without JavaScript | — | Creating a category works, but with JavaScript disabled the success message isn't shown (React's form-state matching with many forms on one page). Normal browsers with JavaScript show it. | | Automated tests | — | None in the repo yet (the verification in §10 used throwaway scripts). | ---

9. Troubleshooting

| Symptom | Cause / Fix | | :--- | :--- | | `RSA public key is not available client side (option serverRsaPublicKeyFile not set)` | You are connecting to a **MySQL 8** server (its `caching_sha2_password` auth) with a MariaDB client/driver. Point the client and `DATABASE_URL` at the **MariaDB** port. On machines with both installed, MySQL 8 usually owns `3306` — check MariaDB's `my.ini` for its port. | | Prisma `P1000 Authentication failed` | Wrong port/server (see above) or the `sporeerp` user doesn't exist — run the SQL in §2.1. | | `Unknown authentication plugin 'auth_gssapi_client'` | MariaDB on Windows replies this way when the user doesn't exist. Create the user (§2.1). | | Auth.js `UntrustedHost` on `next start` | Set `AUTH_TRUST_HOST=true`. | | Build error "encountered the unstable value `new Date()`" / missing Suspense | Cache Components rule — see §6.3. | | Dev error "encountered the unstable value `crypto.getRandomValues()`" at `<UserNav />` | Auth.js generates random values before reading cookies. Always read the session through `getCurrentUser()` (it calls `await connection()` first) — never call `auth()` directly in components. | | Reset/invite link "not received" | Expected — check the server console for `[mail]` output. | | `next build` fails with Prisma connection errors | The homepage/catalog are prerendered from the database at build time — make sure `DATABASE_URL` is reachable during the build. | | Uploaded images 404 after a deploy | `storage/uploads` wasn't carried over — it's gitignored runtime data; mount and back it up like the database. | | Storefront shows old stock or prices | The code that changed catalog data didn't call `updateTag("catalog")` / `revalidateTag("catalog", "max")` (see §7.4). | | Server Action fails with "Body exceeded … limit" | The upload is bigger than 22 MB in total; raise both limits in `next.config.ts` together. | ---

10. Verification Record (2026-10-14)

Run against a production build (`next build` + `next start`) on MariaDB 11.6, after `prisma migrate deploy` + `db:seed` on an empty database. **Database:** both migrations applied; `prisma migrate diff --from-url $DATABASE_URL --to-schema-datamodel prisma/schema.prisma --exit-code` → *No difference detected* (migrations and schema are in sync). Seed → 24 permissions, 6 roles, founder admin. `tsc --noEmit`, `eslint`, `next build` all clean. **End-to-end HTTP checks: 36 / 36 passed.** Forms were submitted the way a no-JavaScript browser submits them (React's progressive-enhancement fields), so the real Server Actions ran. | Area | Verified | | :--- | :--- | | Proxy | Anonymous `/admin/*` → `/login?callbackUrl=…`; anonymous `/api/v1/*` → `401` JSON; customer on `/admin` → `/unauthorized`; non-founder on `/admin/settings` → `?error=restricted` | | Health | `GET /api/health` → `200`, `database: connected` | | Register | JSON API `201`; duplicate email `409 EMAIL_TAKEN`; register form signs the user in and lands on `/account` | | Login | Wrong password rejected; session cookie set; `/api/auth/session` returns `id` + `roles` | | Profile | Profile update; change password rejects wrong current password; new password works | | Reset | Forgot-password returns identical `200` for known/unknown emails; reset link works once, then reports invalid; new password works | | Employees | Founder invites staff → invite link → staff sets password and signs in; Production Staff can open the dashboard but gets `/unauthorized` on `/admin/employees` | | Live RBAC | Promoting the signed-in staff member to `FARM_MANAGER` opens `/admin/employees` **without re-login**; deactivating locks the **same session** out on the next request and blocks new sign-ins | | Guards | Founder attempting to strip their own `FOUNDER_ADMIN` role is rejected | | Audit | `EMPLOYEE_INVITED`, `ROLES_ASSIGNED`, `EMPLOYEE_DEACTIVATED` rows written | Test accounts were deleted afterwards; the database holds only seed data.

Sprint 3 verification

Fresh replay: all **3 migrations** applied to a new empty database → `migrate diff` *No difference detected*. `tsc`, `eslint` and `next build` clean. **Catalog end-to-end checks: 48 / 48 passed** (two consecutive runs), and the Sprint 2 suite re-run afterwards: **36 / 36 passed** (no regressions). | Area | Verified | | :--- | :--- | | Admin | Products and categories screens render; seeded taxonomy present; category created with slug | | Validation | Negative price rejected **and typed values kept**; a non-image file named `.jpg` rejected by content | | Create | Product with 2 uploaded images → SKU `…-OYS-BLU-1KG` generated, unique slug, opening stock, Restaurant tier, featured | | Uploads | Served as `image/png` with immutable caching; `..%2F` traversal → 404; `/_next/image` optimizes them | | Storefront | Listing, variety filter, category + in-stock filter, product page, homepage (featured + variety + live kg) | | SEO | JSON-LD `Product` with `InStock` offer and exact price; OpenGraph + canonical; sitemap lists visible products only; robots disallows `/admin` | | API | Public `GET`; `POST` 401 anonymous / 403 customer / 201 founder (SKU `…-200G`) / 409 duplicate SKU; invalid query 422 | | Notify me | Zero-stock product shows "Out of stock" + form; request stored; repeating it is idempotent | | Pricing rule | Farm Manager sees locked prices; price change rejected; non-price edit saved; Founder price change saved | | Cache | Storefront and API show edits immediately after saving (`updateTag`) | | Images | Removing an image and choosing a new main image works; a file still referenced by another product is **not** deleted | | Archive | Archived product disappears from

listing, API and sitemap; its URL shows "not found" + `noindex` | | RBAC | Customer blocked from `/admin/products`; Farm Manager can manage categories | Two bugs were found by these tests and fixed before the final runs: a failed product save could delete image files it had only *referenced* (API image URLs), and removing an image deleted the file even when another product still used it. > **Gotcha seen during testing:** a `redirect()` from the DAL inside a page that is already streaming (behind `loading.tsx`) arrives as HTTP `200` with a `<meta http-equiv="refresh">` to the target, not a `307`. Browsers follow it normally. Automated tests and API clients must not treat `200` as "allowed" on pages; JSON APIs should return explicit `401`/`403` envelopes instead of calling `redirect()`. ---

11. How to Add a Protected Feature (recipe)

1. **Permission:** if needed, add a code to `PERMISSIONS`, `PERMISSION_DESCRIPTIONS` and the relevant roles in `ROLE_PERMISSIONS` (`lib/auth/permissions.ts`), then `npm run db:seed`. 2. **Schema:** edit `prisma/schema.prisma` → `npm run prisma:migrate -- --name <change>`. 3. **Validation:** add a Zod schema in `lib/validations/<module>.ts`. 4. **Service:** add functions in `services/<module>.service.ts`; throw `ServiceError` for expected failures; log mutations with `AuditService.logEvent` (pass the transaction client when inside `db.$transaction`). 5. **Page:** `app/admin/<module>/page.tsx` → first line `await requirePermission(PERMISSIONS.X, "/admin/<module>")`. 6. **Server Action:** **`"use server"` file next to the page; call `requirePermission` **inside each action**, validate, call the service, `revalidatePath`, return `FormState`. 7. **JSON API (if needed):** `app/api/v1/<module>/route.ts` (already behind the proxy's 401 — add public read endpoints to `isPublicApi` in `proxy.ts`); check permissions with `authorizeApi` (step 10); respond with `apiSuccess` / `apiErrorFrom`. 8. **Nav:** add the link in `components/auth/admin-nav.tsx` with its `visible: hasPermission(...)` condition. 9. **Cache:** if the feature changes anything the storefront shows (stock, price, product status), call `updateTag("catalog")` in the Server Action (or `revalidateTag("catalog", "max")` in a route handler). 10. **JSON API permission checks:** use `authorizeApi(PERMISSIONS.X)` from `lib/auth/api.ts` (returns `401`/`403` envelopes) — not the DAL's redirecting `require*` helpers. ---

12. Next Up — Sprint 4 (Cart, Checkout & Orders)

Per DOC-00: persistent cart (server + localStorage, guest and signed-in), slide-out cart drawer with delivery fee and free-delivery threshold, 3-step checkout (address with saved-address picker → delivery slot → payment method), order review with **stock verification**, `Order` / `OrderItem` / `OrderStatusHistory` written atomically, the order state machine (`PENDING → CONFIRMED → PROCESSING → PACKED → OUT_FOR_DELIVERY → DELIVERED`, or `CANCELLED` / `REFUNDED`), admin orders Kanban + table, packing slip / invoice PDF. Already in place for it: the product page quantity stepper and disabled **Add to cart** button, `minOrderQty`, B2B `pricingTiers` (apply by `customers.customer_type`), `allowBackorder`, `inventory_items.reserved_stock` (reserve on order, release on cancel) and the `catalog` cache tag (invalidate whenever stock changes). Open gaps from earlier sprints are listed in §8 (email provider, rate limiting, tests).